Medsien

Load Balancer Simulation Assignment

Overview

You are asked to design and implement a deterministic load balancer simulation using Python. The system must distribute incoming requests across heterogeneous servers (cluster) while respecting: CPU capacity, memory limits and rate limits.

This assignment evaluates system design, architectural thinking, correctness, and clarity of reasoning. You are expected to build a clean, extensible backend system and a server management dashboard.

The use of AI assistants and tools is required and will be considered as part of the evaluation. Candidates must document how AI was used, including prompts and verification steps.

Technical Constraints

Containerization

The entire project must be fully dockerized.

- Use Docker and Docker Compose.
- All services must run via docker-compose up.
- If your architecture requires a database or Redis, they must also run as containers.
- No manual local dependency setup should be required beyond Docker.

Your repository must include:

- Dockerfile(s)
- docker-compose.yml
- Clear instructions to run the system using Docker only.

Backend Requirements

- Preferred language: Python
- Acceptable frameworks: FastAPI, Flask, Django
- JavaScript/Node.js backends are also acceptable if properly justified.
- The backend must expose APIs required by the dashboard and simulation engine.

Frontend Requirements

- Frontend framework is your choice.
- React is preferred but not mandatory.
- The server management dashboard is mandatory.
- Simulation visualization UI is optional.

Determinism

The simulation must be deterministic.

Given the same:

- servers.json
- requests.csv
- configuration

The system must produce the same run.jsonl output. If multiple scheduling decisions are possible at the same tick, you must define and document a deterministic tie-breaking strategy.

Provided Data

The provided dataset is for testing and development purposes only and is provided as an example. During the interview, additional or modified test data may be introduced.

If you consider making modifications in the data schema, please explicitly document your new models and reasoning. Also update validate_run.py accordingly.

1. servers.json

Defines the initial server configuration.
Each server contains:

- id
- cpu_units_per_tick
- mem_mb
- rate_limit_per_sec

You must support creating, editing, and deleting servers via the dashboard.

2. requests.csv

Defines incoming requests over time.

Columns:

- t (arrival tick)
- request_id
- work_units
- mem_mb

Requests arrive at the specified tick. Based on the capacity of the worker(server), it occupies it for work_units/cpu_units_per_tick ticks (seconds).

3. Output: run.jsonl

Your simulation must generate a file named run.jsonl representing the full simulation trace.

Each line must be a JSON object.

Supported events:

- REQUEST_ARRIVED
- REQUEST_STARTED
- REQUEST_FINISHED
- REQUEST_DROPPED (optional)

Example:

{"t":0,"event":"REQUEST_ARRIVED","request_id":"r1"}
{"t":0,"event":"REQUEST_STARTED","request_id":"r1","server_id":"s1"}
{"t":2,"event":"REQUEST_FINISHED","request_id":"r1","server_id":"s1"}

The output must be deterministic for the same input.

4. Validator

A validation script is provided: validate_run.py
It will:

- Replay your simulation output
- Verify lifecycle correctness
- Validate resource constraints
- Enforce rate limits
- Compute summary metrics

Your run.jsonl must pass validation.

Core Requirements

1. Load Balancer Engine

You must:

- Simulate discrete time (ticks). (1tick = 1 second)
- Assign requests to servers.
- Respect CPU, memory, and rate constraints.
- Ensure no request runs on multiple servers simultaneously.
- Ensure deterministic execution.
- Requests that cannot be scheduled immediately may either queue or be dropped.

Your strategy must be documented.

- You may choose your own scheduling strategy and rate limiting implementation.
- Document your assumptions clearly.

2. Server Management Dashboard

You must implement a dashboard that allows:

- Viewing all servers
- Adding servers
- Editing servers
- Deleting servers

Changes must affect future simulation runs.
Note: Frontend polish is not evaluated beyond functionality.

3. Simulation Execution

Your project must provide:
- A way to trigger simulation runs
- A generated run.jsonl
- Instructions explaining how to run the project
- Re-running the same simulation must overwrite or version run.jsonl deterministically.

PS: Events at time t describe the system state after processing tick t-1 (or at the start of tick t).

4. Execution Model

- A server may execute multiple requests concurrently, as long as:
  - Total reserved memory does not exceed mem_mb
  - Per-tick CPU allocation does not exceed cpu_units_per_tick
  - Rate limits are respected: Rate limit is enforced as: max REQUEST_STARTED events per server per tick ≤ rate_limit_per_sec
- Memory is reserved for the full duration of a request's execution.
- Each request has work_units. At each tick, a server distributes its cpu_units_per_tick evenly across all currently running requests.
- A request's remaining work is reduced by its allocated CPU share each tick.
- A request finishes when its remaining work reaches zero or below.
- All CPU allocation and completion checks must occur in a deterministic order. If multiple completion or scheduling decisions are possible within the same tick, the tie-breaking strategy must be explicitly documented.

5. Optional (Bonus)

Bonus features are optional. Candidates may choose zero or more to implement based on their priorities and available time.

- Simulation visualization UI
- Multiple, interchangeable scheduling strategies
- Performance metrics dashboard (per server & cluster-wide)
- An auto-scaling decision module that increases or decreases server capacity based on simulated system metrics (CPU, queue length, error rate).

Submission Requirements

Please submit:

- GitHub repository link
- Clear "How to Run" instructions
- Generated run.jsonl example
- Design documentation explaining:
  - Scheduling approach
  - Rate limiting strategy
  - System architecture
  - Key assumptions and trade-offs
  - Description of how AI is used in this project

Evaluation Criteria

We will evaluate:

- Correctness (must pass validator)
- System architecture quality
- Code organization and clarity
- Handling of overload scenarios
- Quality of documentation
- Ability to extend during live discussion
- Appropriate and effective AI-tool usage

Timeline

You have **one week** to complete the assignment.

If any requirement is unclear, you are encouraged to email us (eng-leads@medsien.com) for further clarification.